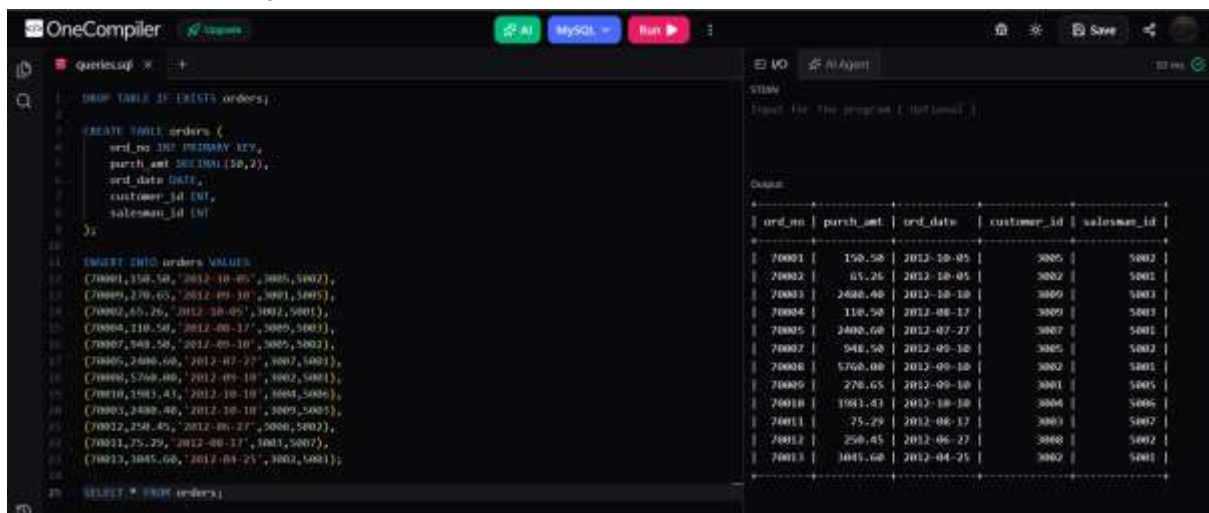


SQL PRACTICE NOTES - ORDERS TABLE

STEP 1 — Create Orders Table + Insert Data

Create and Populate Orders Table

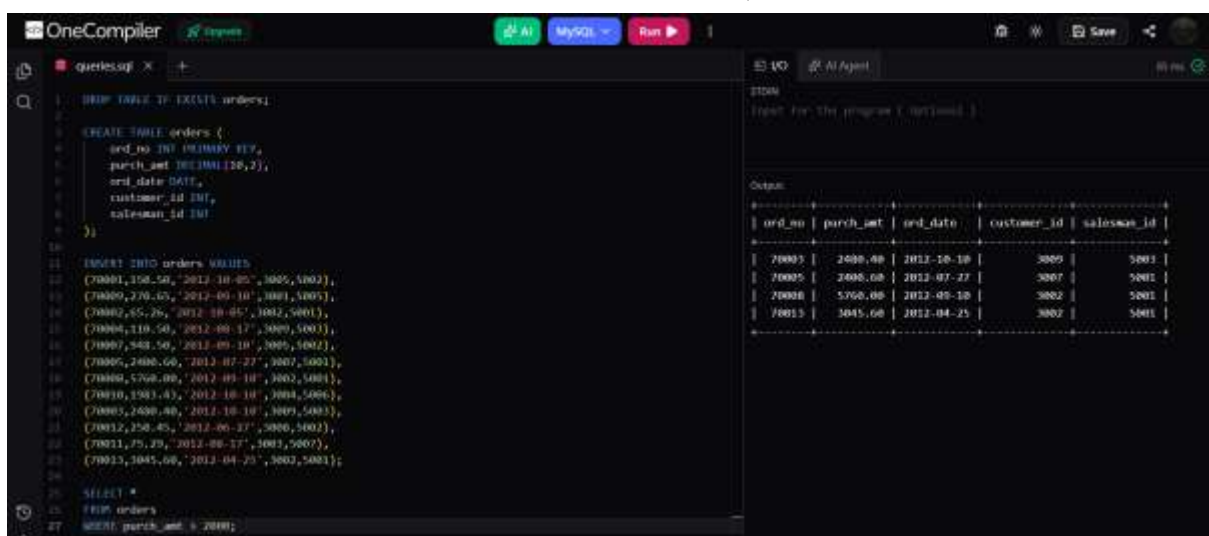


```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.65,'2012-09-10',3001,5005),
14 (70002,45.26,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-08-17',3009,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70006,5760.00,'2012-09-10',3002,5001),
19 (70008,5760.00,'2012-09-10',3002,5001),
20 (70010,1981.43,'2012-10-10',3004,5006),
21 (70003,2400.40,'2012-10-10',3009,5003),
22 (70012,250.45,'2012-06-27',3000,5002),
23 (70011,75.29,'2012-06-17',3003,5007),
24 (70013,3045.60,'2012-04-25',3002,5001);
25
26 SELECT * FROM orders;
```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70002	45.26	2012-10-05	3002	5001
70003	2400.40	2012-10-10	3009	5003
70004	110.50	2012-08-17	3009	5003
70005	2400.60	2012-07-27	3007	5001
70006	940.50	2012-09-10	3005	5002
70007	5760.00	2012-09-10	3002	5001
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70010	1981.43	2012-10-10	3004	5006
70011	75.29	2012-06-17	3003	5007
70012	250.45	2012-06-27	3000	5002
70013	3045.60	2012-04-25	3002	5001

Q1 — WHERE

WHERE — Find Orders Above \$2,000

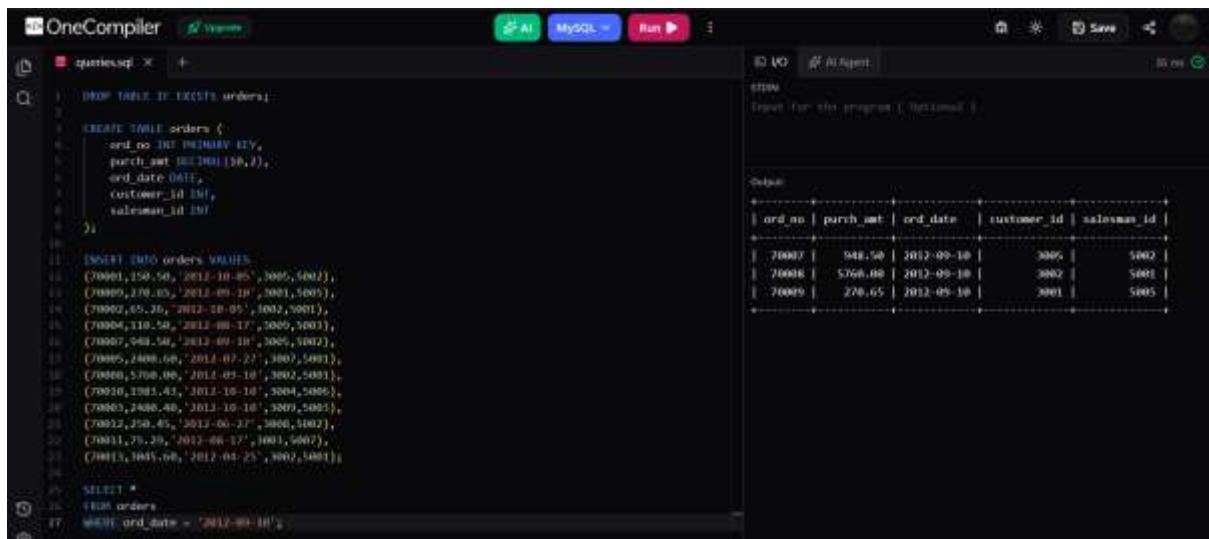


```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.65,'2012-09-10',3001,5005),
14 (70002,45.26,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-08-17',3009,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70006,5760.00,'2012-09-10',3002,5001),
19 (70008,5760.00,'2012-09-10',3002,5001),
20 (70010,1981.43,'2012-10-10',3004,5006),
21 (70003,2400.40,'2012-10-10',3009,5003),
22 (70012,250.45,'2012-06-27',3000,5002),
23 (70011,75.29,'2012-06-17',3003,5007),
24 (70013,3045.60,'2012-04-25',3002,5001);
25
26 SELECT *
27 FROM orders
28 WHERE purch_amt > 2000;
```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70005	2400.60	2012-07-27	3007	5001
70006	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001

Q2 — WHERE

WHERE — Find Orders Placed on 2012-09-10



The screenshot shows the OneCompiler MySQL IDE. The left pane contains a SQL script with the following code:

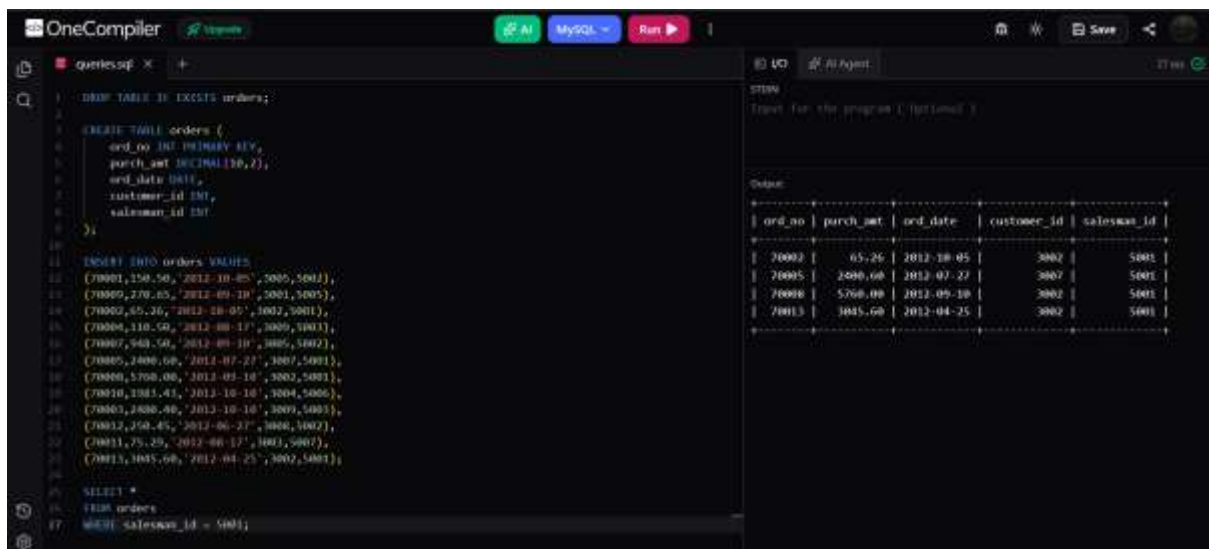
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.85,'2012-09-10',3001,5005),
14 (70002,45.20,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-08-17',3009,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70008,5700.00,'2012-09-10',3002,5001),
19 (70010,13015.41,'2012-10-10',3004,5006),
20 (70003,2400.40,'2012-10-10',3009,5005),
21 (70012,250.45,'2012-06-17',3008,5002),
22 (70011,75.25,'2012-06-17',3003,5002),
23 (70013,3045.60,'2012-04-25',3002,5001);
24
25 SELECT *
26 FROM orders
27 WHERE ord_date = '2012-09-10';
```

The right pane shows the output of the query as a table:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70007	940.50	2012-09-10	3005	5002
70008	5700.00	2012-09-10	3002	5001
70009	270.85	2012-09-10	3001	5005

Q3 — WHERE

WHERE — Find Orders Handled by Salesman 5001



The screenshot shows the OneCompiler MySQL IDE. The left pane contains a SQL script with the following code:

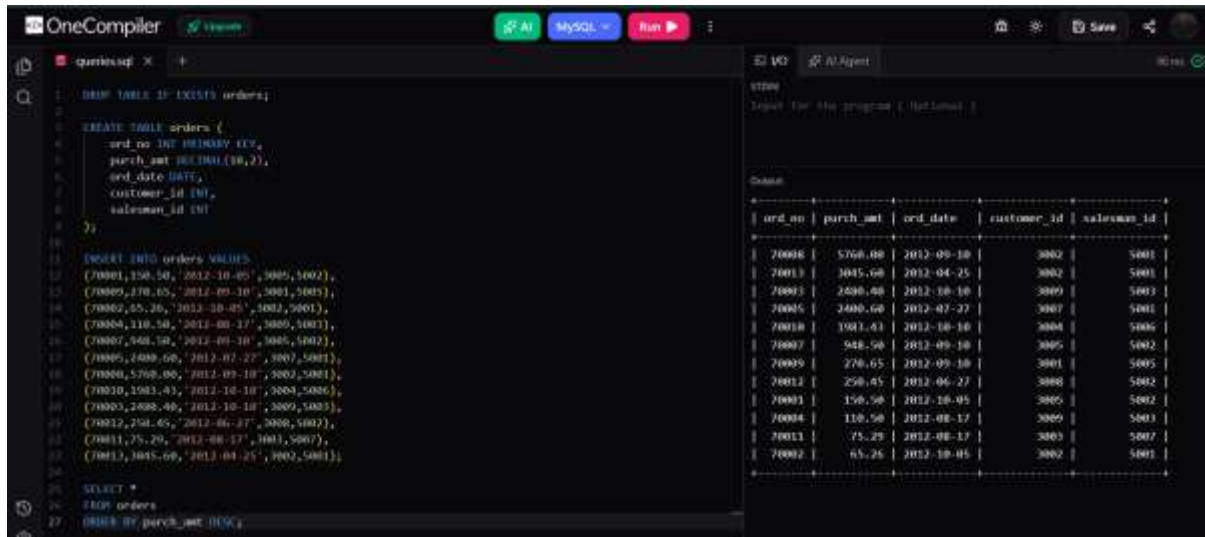
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.85,'2012-09-10',3001,5005),
14 (70002,45.20,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-08-17',3009,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70008,5700.00,'2012-09-10',3002,5001),
19 (70010,13015.41,'2012-10-10',3004,5006),
20 (70003,2400.40,'2012-10-10',3009,5005),
21 (70012,250.45,'2012-06-17',3008,5002),
22 (70011,75.25,'2012-06-17',3003,5002),
23 (70013,3045.60,'2012-04-25',3002,5001);
24
25 SELECT *
26 FROM orders
27 WHERE salesman_id = 5001;
```

The right pane shows the output of the query as a table:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70002	45.20	2012-10-05	3002	5001
70005	2400.60	2012-07-27	3007	5001
70008	5700.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001

Q4 — ORDER BY

ORDER BY — Rank Orders from Highest to Lowest Amount

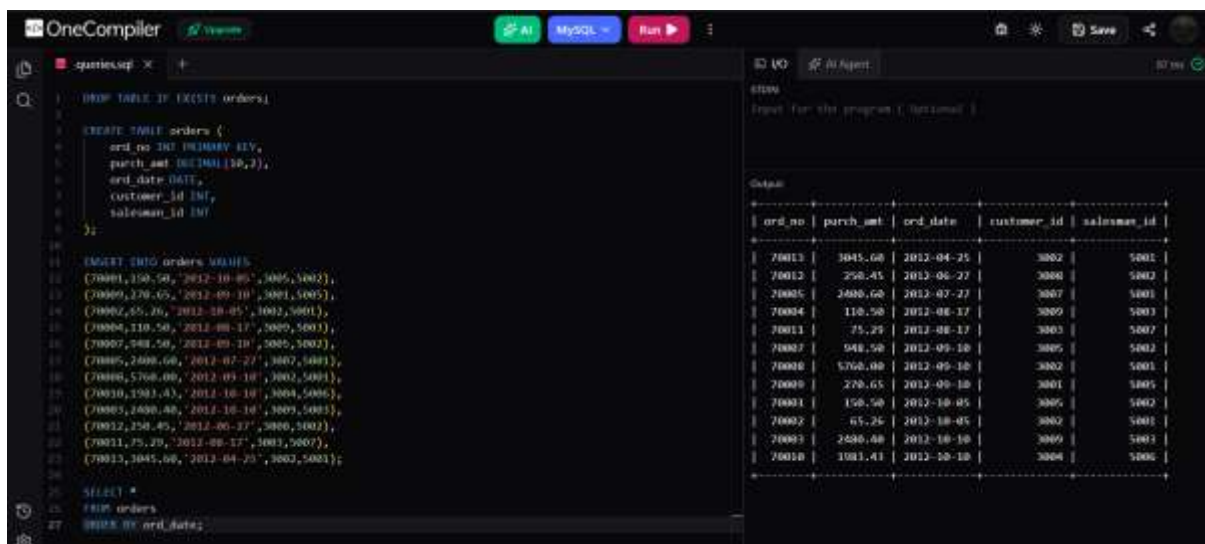


```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.65,'2012-09-10',3001,5003),
14 (70002,45.26,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-08-17',3009,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70000,5768.00,'2012-09-10',3002,5001),
19 (70010,1081.43,'2012-10-10',3004,5006),
20 (70003,2600.00,'2012-10-10',3009,5003),
21 (70012,250.45,'2012-06-27',3000,5002),
22 (70011,75.29,'2012-08-17',3003,5007),
23 (70013,3045.00,'2012-04-25',3002,5001);
24
25 SELECT *
26 FROM orders
27 ORDER BY purch_amt DESC;
```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70006	5768.00	2012-09-10	3002	5001
70013	3045.00	2012-04-25	3002	5001
70003	2600.00	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70010	1081.43	2012-10-10	3004	5006
70007	940.50	2012-09-10	3005	5002
70009	270.65	2012-09-10	3001	5003
70012	250.45	2012-06-27	3000	5002
70001	150.50	2012-10-05	3005	5002
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70002	45.26	2012-10-05	3002	5001

Q5 — ORDER BY

ORDER BY — Display Oldest Orders First

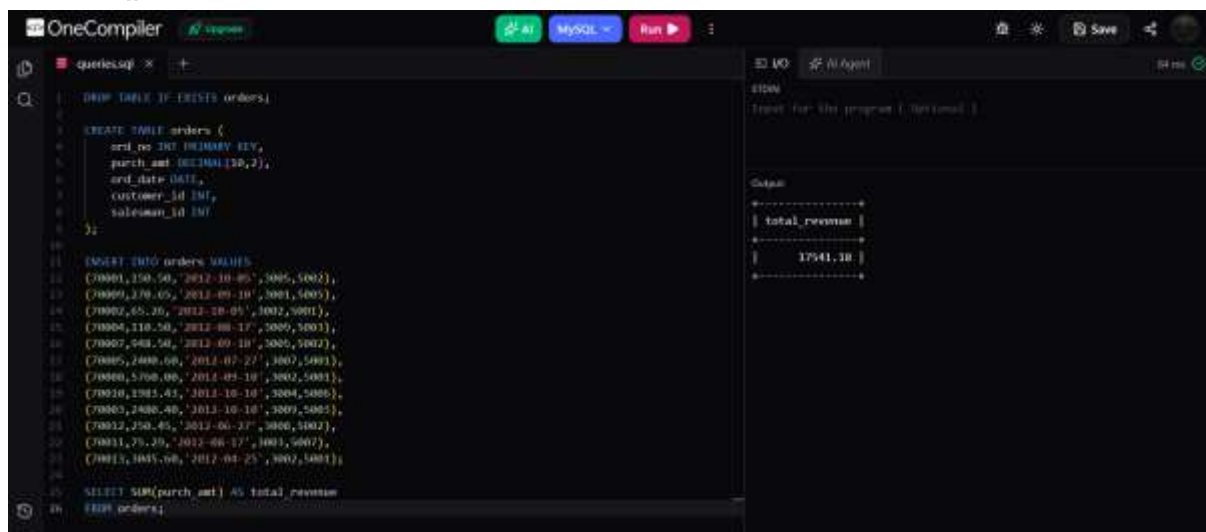


```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.65,'2012-09-10',3001,5003),
14 (70002,45.26,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-08-17',3009,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70000,5768.00,'2012-09-10',3002,5001),
19 (70010,1081.43,'2012-10-10',3004,5006),
20 (70003,2600.00,'2012-10-10',3009,5003),
21 (70012,250.45,'2012-06-27',3000,5002),
22 (70011,75.29,'2012-08-17',3003,5007),
23 (70013,3045.00,'2012-04-25',3002,5001);
24
25 SELECT *
26 FROM orders
27 ORDER BY ord_date;
```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70013	3045.00	2012-04-25	3002	5001
70012	250.45	2012-06-27	3000	5002
70005	2400.60	2012-07-27	3007	5001
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70007	940.50	2012-09-10	3005	5002
70000	5768.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5003
70001	150.50	2012-10-05	3005	5002
70002	45.26	2012-10-05	3002	5001
70003	2600.00	2012-10-10	3009	5003
70010	1081.43	2012-10-10	3004	5006

Q6 — SUM()

SUM() — Find Total Revenue Generated



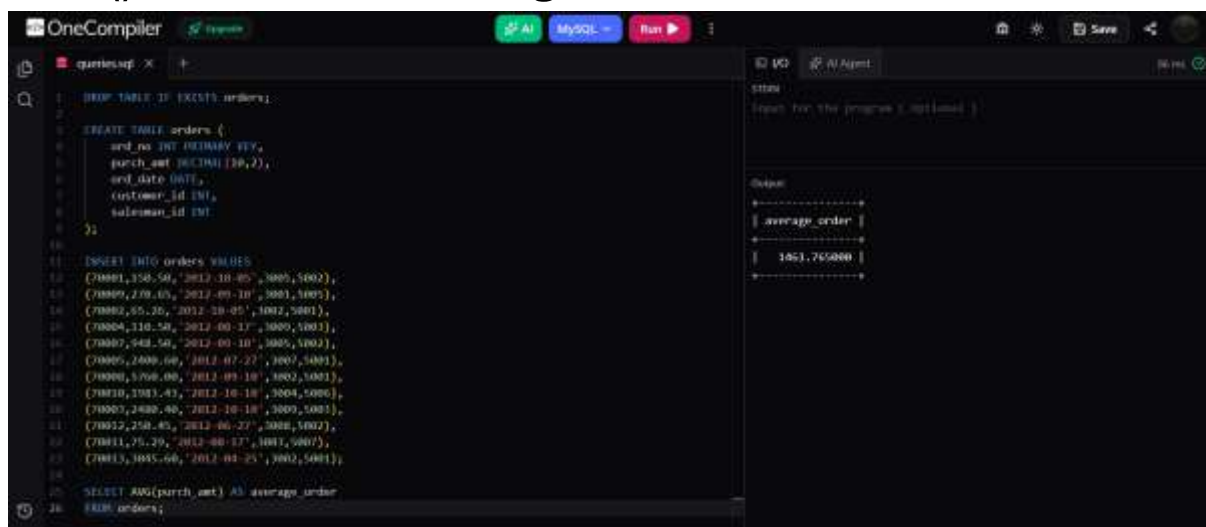
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.65,'2012-09-10',3001,5005),
14 (70002,45.20,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-08-17',3000,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70000,5700.00,'2012-09-10',3002,5001),
19 (70010,1303.43,'2012-10-10',3004,5006),
20 (70003,2400.40,'2012-10-10',3009,5005),
21 (70012,250.45,'2012-06-27',3008,5002),
22 (70011,75.29,'2012-06-17',3001,5007),
23 (70013,3045.60,'2012-04-25',3002,5001);
24
25 SELECT SUM(purch_amt) AS total_revenue
26 FROM orders;
```

Output

total_revenue
17941.10

Q7 — AVG()

AVG() — Find the Average Order Value



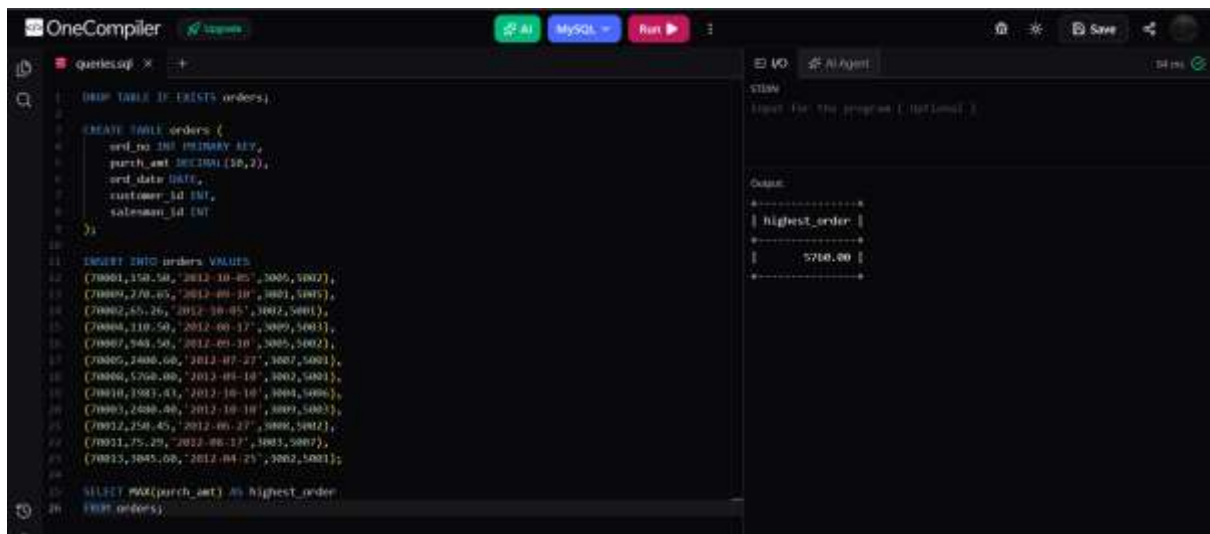
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.65,'2012-09-10',3001,5005),
14 (70002,45.20,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-08-17',3000,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70000,5700.00,'2012-09-10',3002,5001),
19 (70010,1303.43,'2012-10-10',3004,5006),
20 (70003,2400.40,'2012-10-10',3009,5005),
21 (70012,250.45,'2012-06-27',3008,5002),
22 (70011,75.29,'2012-06-17',3001,5007),
23 (70013,3045.60,'2012-04-25',3002,5001);
24
25 SELECT AVG(purch_amt) AS average_order
26 FROM orders;
```

Output

average_order
1453.750000

Q8 — MAX()

MAX() — Find the Highest Order Amount



The screenshot shows the OneCompiler MySQL IDE. The left pane contains a SQL script with the following code:

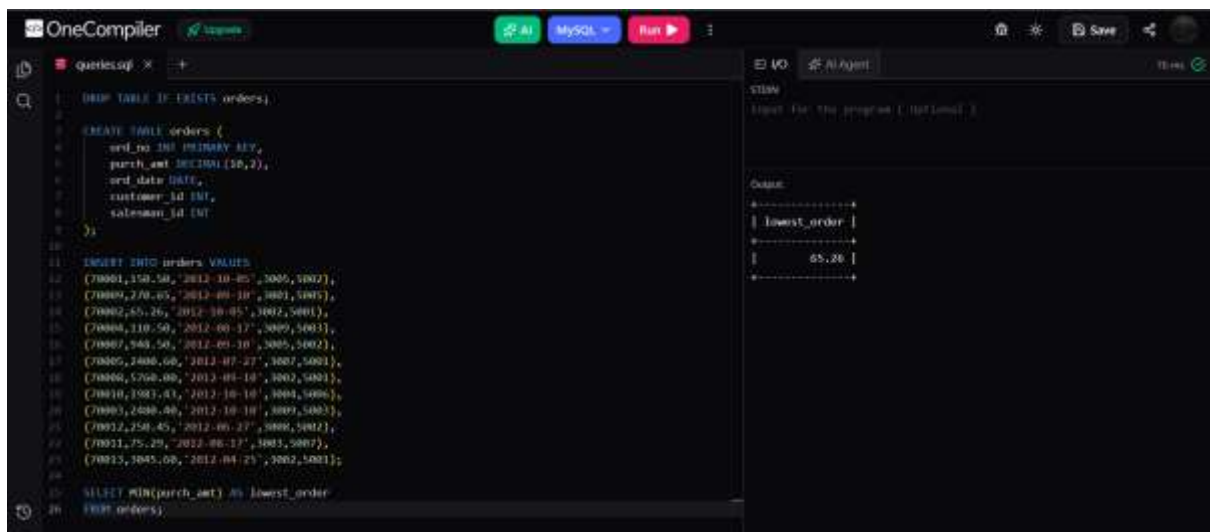
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,770.05,'2012-09-10',3001,5005),
14 (70002,65.26,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-09-17',3009,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70006,5700.00,'2012-09-10',3002,5001),
19 (70010,1903.03,'2012-10-10',3004,5006),
20 (70003,2400.00,'2012-10-10',3009,5003),
21 (70012,250.45,'2012-09-17',3008,5002),
22 (70011,75.29,'2012-09-17',3003,5007),
23 (70013,3045.00,'2012-04-25',3002,5001);
24
25 SELECT MAX(purch_amt) AS highest_order
26 FROM orders;
```

The right pane shows the output of the query:

```
Output:
+-----+
| highest_order |
+-----+
|          5700.00 |
+-----+
```

Q9 — MIN()

MIN() — Find the Lowest Order Amount



The screenshot shows the OneCompiler MySQL IDE. The left pane contains a SQL script with the following code:

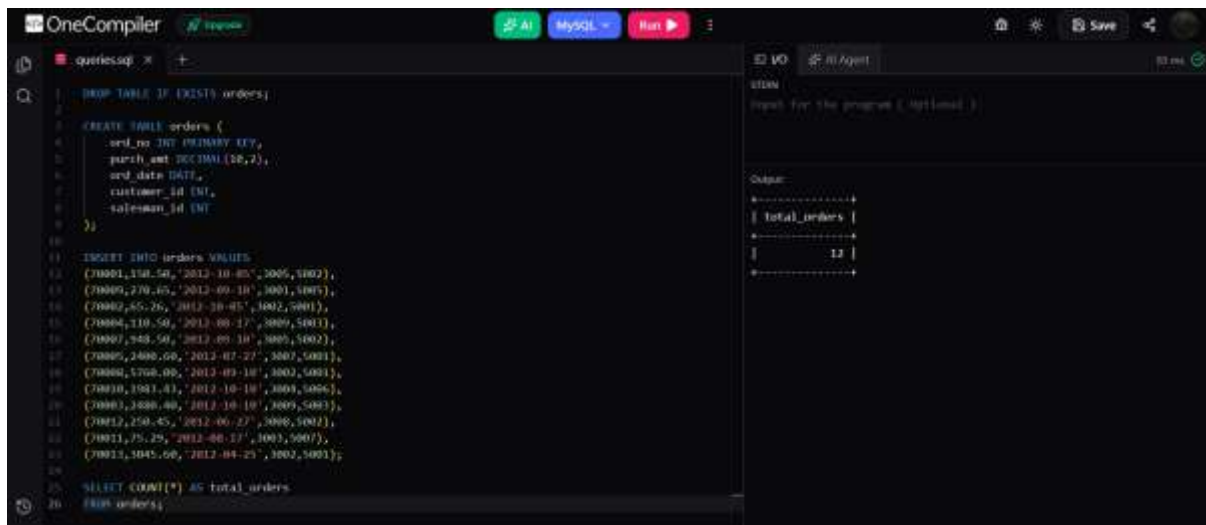
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,770.05,'2012-09-10',3001,5005),
14 (70002,65.26,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-09-17',3009,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70006,5700.00,'2012-09-10',3002,5001),
19 (70010,1903.03,'2012-10-10',3004,5006),
20 (70003,2400.00,'2012-10-10',3009,5003),
21 (70012,250.45,'2012-09-17',3008,5002),
22 (70011,75.29,'2012-09-17',3003,5007),
23 (70013,3045.00,'2012-04-25',3002,5001);
24
25 SELECT MIN(purch_amt) AS lowest_order
26 FROM orders;
```

The right pane shows the output of the query:

```
Output:
+-----+
| lowest_order |
+-----+
|          65.26 |
+-----+
```

Q10 — COUNT()

COUNT() — Find the Total Number of Orders



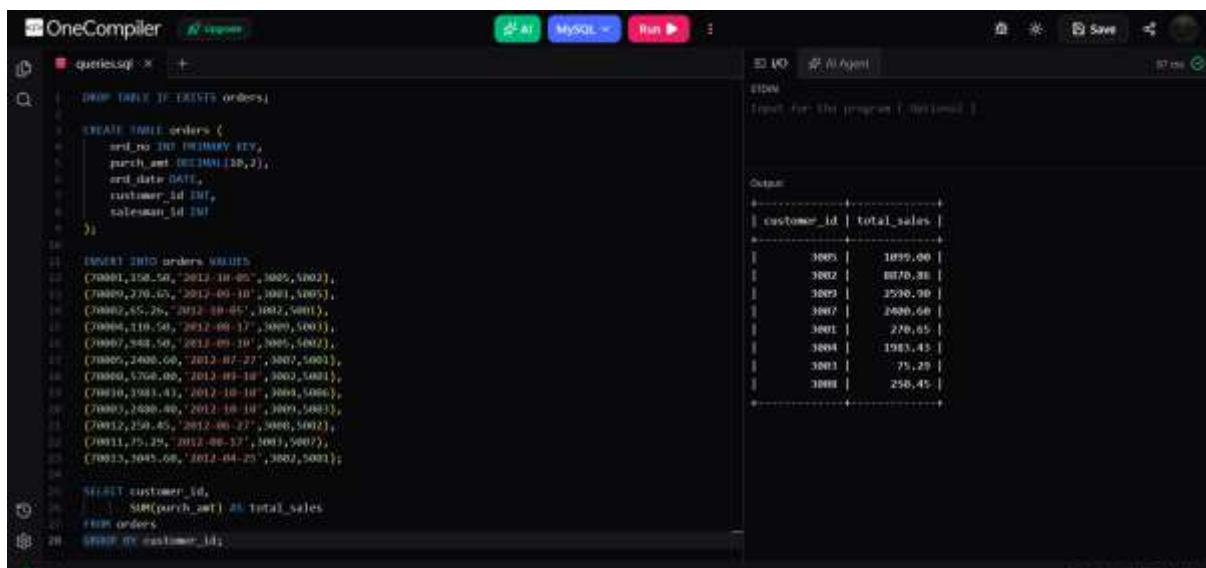
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.85,'2012-09-10',3001,5005),
14 (70002,45.26,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-09-17',3009,5003),
16 (70007,543.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70008,5768.00,'2012-09-10',3002,5001),
19 (70010,1983.43,'2012-10-10',3009,5006),
20 (70003,2486.40,'2012-10-10',3009,5003),
21 (70012,250.45,'2012-06-27',3000,5002),
22 (70011,75.29,'2012-06-17',3003,5007),
23 (70013,3045.60,'2012-04-25',3002,5001);
24
25 SELECT COUNT(*) AS total_orders
26 FROM orders;
```

Output:

total_orders
13

Q11 — GROUP BY

GROUP BY — Find Total Sales for Each Customer



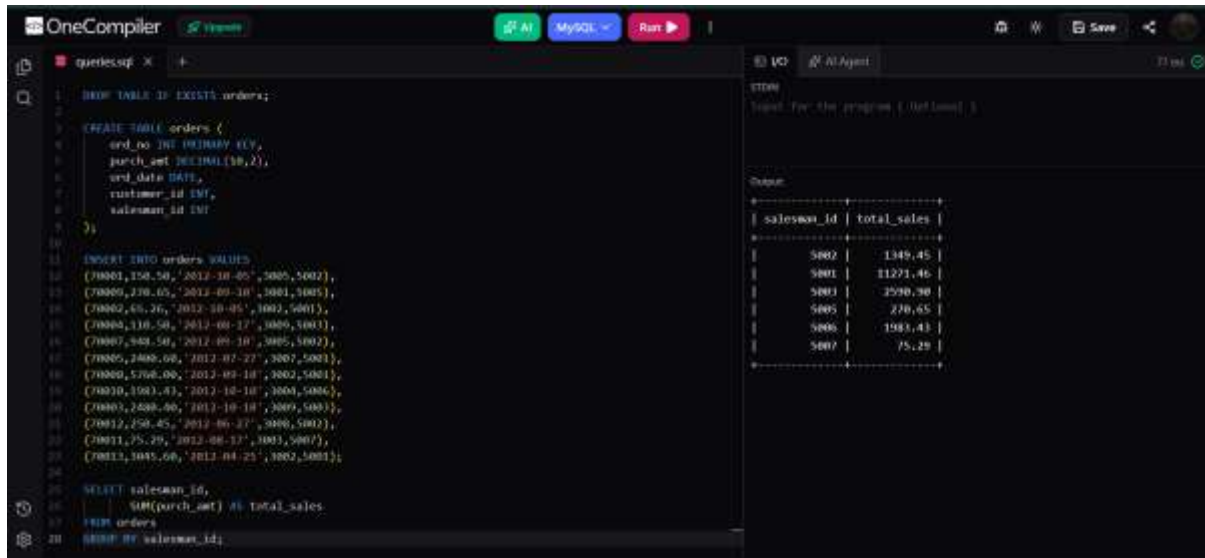
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.85,'2012-09-10',3001,5005),
14 (70002,45.26,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-09-17',3009,5003),
16 (70007,543.50,'2012-09-10',3005,5002),
17 (70005,2400.60,'2012-07-27',3007,5001),
18 (70008,5768.00,'2012-09-10',3002,5001),
19 (70010,1983.43,'2012-10-10',3009,5006),
20 (70003,2486.40,'2012-10-10',3009,5003),
21 (70012,250.45,'2012-06-27',3000,5002),
22 (70011,75.29,'2012-06-17',3003,5007),
23 (70013,3045.60,'2012-04-25',3002,5001);
24
25 SELECT customer_id,
26        SUM(purch_amt) AS total_sales
27 FROM orders
28 GROUP BY customer_id;
```

Output:

customer_id	total_sales
3005	1099.00
3002	8076.38
3009	2790.90
3007	2400.60
3001	270.85
3004	1983.43
3003	75.29
3008	250.45

Q12 — GROUP BY

GROUP BY — Find Total Sales Handled by Each Salesman



The screenshot shows the OneCompiler interface with a MySQL query editor on the left and an output window on the right. The query creates an 'orders' table, inserts 13 rows of data, and then uses a GROUP BY clause to calculate the total sales for each salesman.

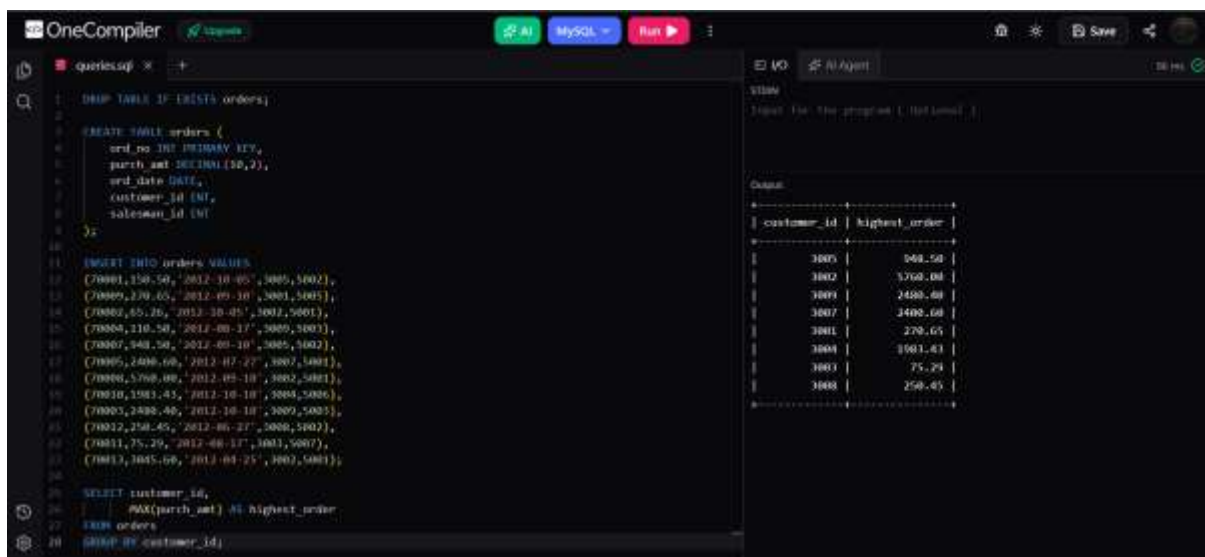
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.65,'2012-09-10',3001,5005),
14 (70002,45.26,'2012-10-05',1002,5001),
15 (70004,110.50,'2012-09-17',3005,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.00,'2012-07-27',3007,5001),
18 (70000,5760.00,'2012-09-10',3002,5001),
19 (70010,1901.43,'2012-10-10',3004,5006),
20 (70003,2400.00,'2012-10-10',3009,5003),
21 (70012,250.45,'2012-06-17',3000,5002),
22 (70011,75.29,'2012-06-17',1003,5007),
23 (70013,1045.00,'2012-04-25',3002,5003);
24
25 SELECT salesman_id,
26        SUM(purch_amt) AS total_sales
27 FROM orders
28 GROUP BY salesman_id;
```

The output window displays the results of the query:

salesman_id	total_sales
5002	1349.45
5001	11221.46
5003	2590.30
5005	270.65
5006	1901.43
5007	75.29

Q13 — GROUP BY + MAX()

GROUP BY + MAX() — Find the Highest Order for Each Customer



The screenshot shows the OneCompiler interface with a MySQL query editor on the left and an output window on the right. The query creates an 'orders' table, inserts 13 rows of data, and then uses a GROUP BY clause along with the MAX() function to find the highest order number for each customer.

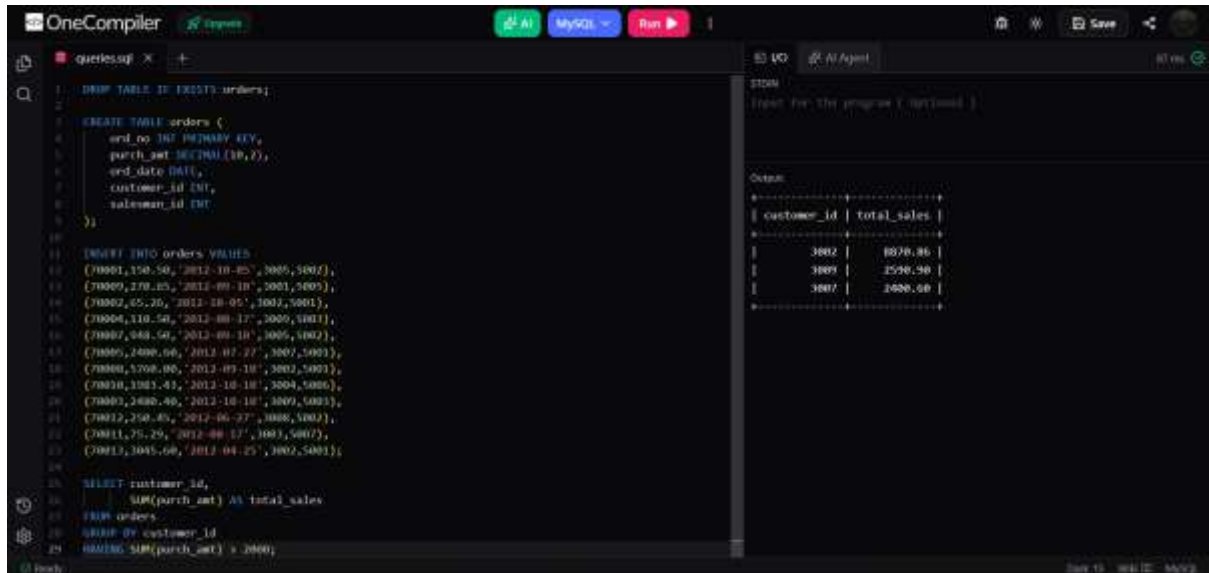
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.65,'2012-09-10',3001,5005),
14 (70002,45.26,'2012-10-05',1002,5001),
15 (70004,110.50,'2012-09-17',3005,5003),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.00,'2012-07-27',3007,5001),
18 (70000,5760.00,'2012-09-10',3002,5001),
19 (70010,1901.43,'2012-10-10',3004,5006),
20 (70003,2400.00,'2012-10-10',3009,5003),
21 (70012,250.45,'2012-06-17',3000,5002),
22 (70011,75.29,'2012-06-17',1003,5007),
23 (70013,1045.00,'2012-04-25',3002,5003);
24
25 SELECT customer_id,
26        MAX(ord_no) AS highest_order
27 FROM orders
28 GROUP BY customer_id;
```

The output window displays the results of the query:

customer_id	highest_order
3005	940.50
3002	5760.00
3009	2400.00
3007	2400.00
3001	270.65
3004	1901.43
3003	75.29
3000	250.45

Q14 — GROUP BY + HAVING

GROUP BY + HAVING — Find Customers with Total Sales Above 2,000



The screenshot shows the OneCompiler IDE with a MySQL database. The code defines an 'orders' table and inserts 13 rows of data. The query uses GROUP BY on customer_id and HAVING to filter for total sales above 2000. The output shows three customers meeting the criteria.

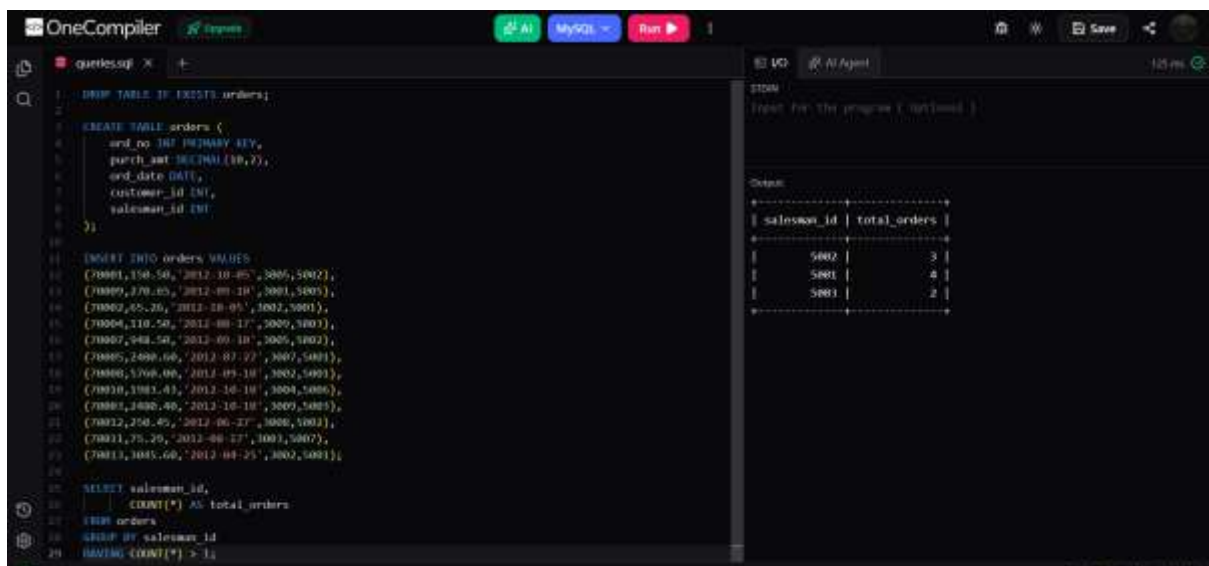
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.85,'2012-09-10',3001,5003),
14 (70002,45.20,'2012-10-05',1002,5001),
15 (70004,110.50,'2012-08-17',3000,5001),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.66,'2012-07-27',3007,5001),
18 (70006,5700.00,'2012-09-10',3002,5001),
19 (70010,1001.41,'2012-10-10',3004,5006),
20 (70003,2400.40,'2012-10-10',3009,5003),
21 (70012,250.45,'2012-06-27',3008,5002),
22 (70011,75.25,'2012-06-27',1003,5007),
23 (70013,1045.60,'2012-04-25',3002,5001);
24
25 SELECT customer_id,
26        SUM(purch_amt) AS total_sales
27 FROM orders
28 GROUP BY customer_id
29 HAVING SUM(purch_amt) > 2000;
```

Output:

customer_id	total_sales
3002	8070.86
3009	2500.90
3007	2400.66

Q15 — GROUP BY + HAVING

GROUP BY + HAVING — Find Salesmen with More Than One Order



The screenshot shows the OneCompiler IDE with the same MySQL database. The query uses GROUP BY on salesman_id and HAVING to filter for more than one order. The output shows three salesmen meeting the criteria.

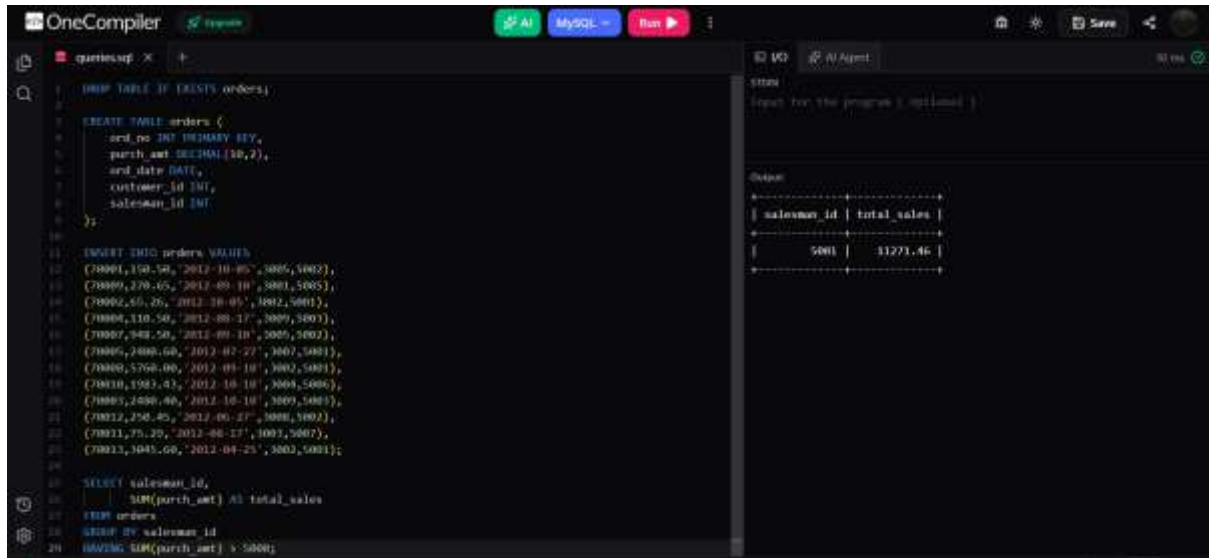
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.85,'2012-09-10',3001,5003),
14 (70002,45.20,'2012-10-05',1002,5001),
15 (70004,110.50,'2012-08-17',3000,5001),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.66,'2012-07-27',3007,5001),
18 (70006,5700.00,'2012-09-10',3002,5001),
19 (70010,1001.41,'2012-10-10',3004,5006),
20 (70003,2400.40,'2012-10-10',3009,5003),
21 (70012,250.45,'2012-06-27',3008,5002),
22 (70011,75.25,'2012-06-27',1003,5007),
23 (70013,1045.60,'2012-04-25',3002,5001);
24
25 SELECT salesman_id,
26        COUNT(*) AS total_orders
27 FROM orders
28 GROUP BY salesman_id
29 HAVING COUNT(*) > 1;
```

Output:

salesman_id	total_orders
5002	3
5001	4
5003	2

Q16 — GROUP BY + HAVING

GROUP BY + HAVING — Find Salesmen with Total Sales Above 5,000



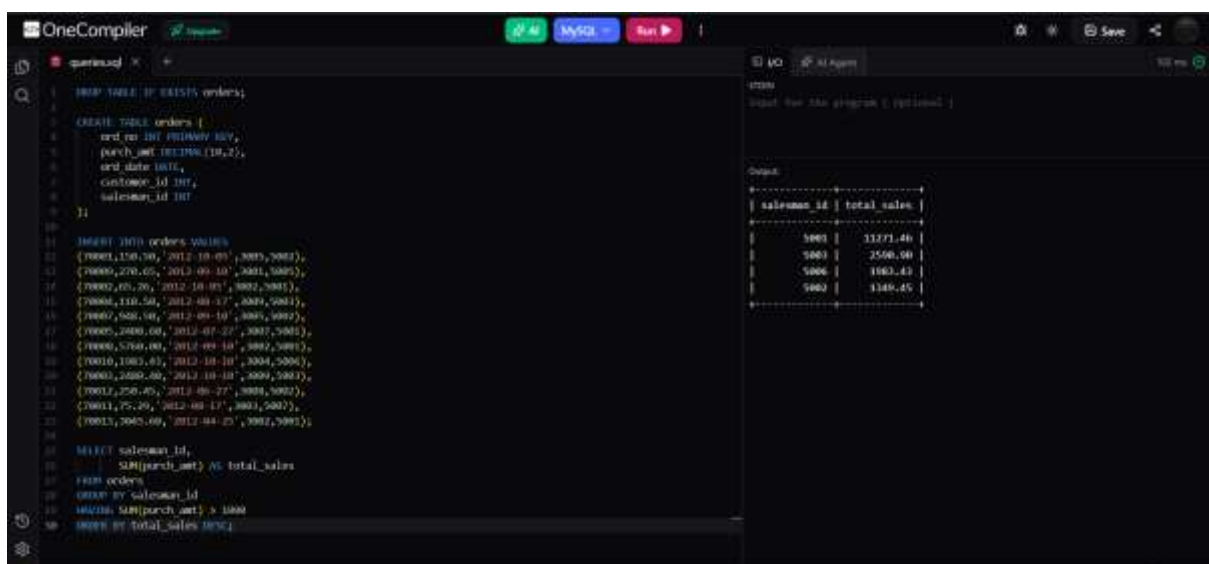
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.05,'2012-09-10',3001,5005),
14 (70002,55.20,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-08-17',3009,5002),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.00,'2012-07-27',3007,5001),
18 (70006,5760.00,'2012-09-10',3002,5001),
19 (70010,1063.43,'2012-10-10',3004,5006),
20 (70003,2400.00,'2012-10-10',3009,5005),
21 (70012,250.45,'2012-06-27',3008,5002),
22 (70011,75.20,'2012-06-27',3003,5007),
23 (70013,3045.00,'2012-04-25',3003,5001);
24
25 SELECT salesman_id,
26        SUM(purch_amt) AS total_sales
27 FROM orders
28 GROUP BY salesman_id
29 HAVING SUM(purch_amt) > 5000;
```

Output:

salesman_id	total_sales
5001	11271.46

Q17 — GROUP BY + HAVING + ORDER BY

GROUP BY + HAVING + ORDER BY — Rank Salesmen with Total Sales Above 1,000



```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 (70001,150.50,'2012-10-05',3005,5002),
13 (70009,270.05,'2012-09-10',3001,5005),
14 (70002,55.20,'2012-10-05',3002,5001),
15 (70004,110.50,'2012-08-17',3009,5002),
16 (70007,940.50,'2012-09-10',3005,5002),
17 (70005,2400.00,'2012-07-27',3007,5001),
18 (70006,5760.00,'2012-09-10',3002,5001),
19 (70010,1063.43,'2012-10-10',3004,5006),
20 (70003,2400.00,'2012-10-10',3009,5005),
21 (70012,250.45,'2012-06-27',3008,5002),
22 (70011,75.20,'2012-06-27',3003,5007),
23 (70013,3045.00,'2012-04-25',3003,5001);
24
25 SELECT salesman_id,
26        SUM(purch_amt) AS total_sales
27 FROM orders
28 GROUP BY salesman_id
29 HAVING SUM(purch_amt) > 1000
30 ORDER BY total_sales DESC;
```

Output:

salesman_id	total_sales
5001	11271.46
5003	2500.00
5006	1063.43
5002	1146.45

Q18 — BETWEEN + HAVING

BETWEEN + HAVING — Find Customers with Sales Between 1,000 and 5,000



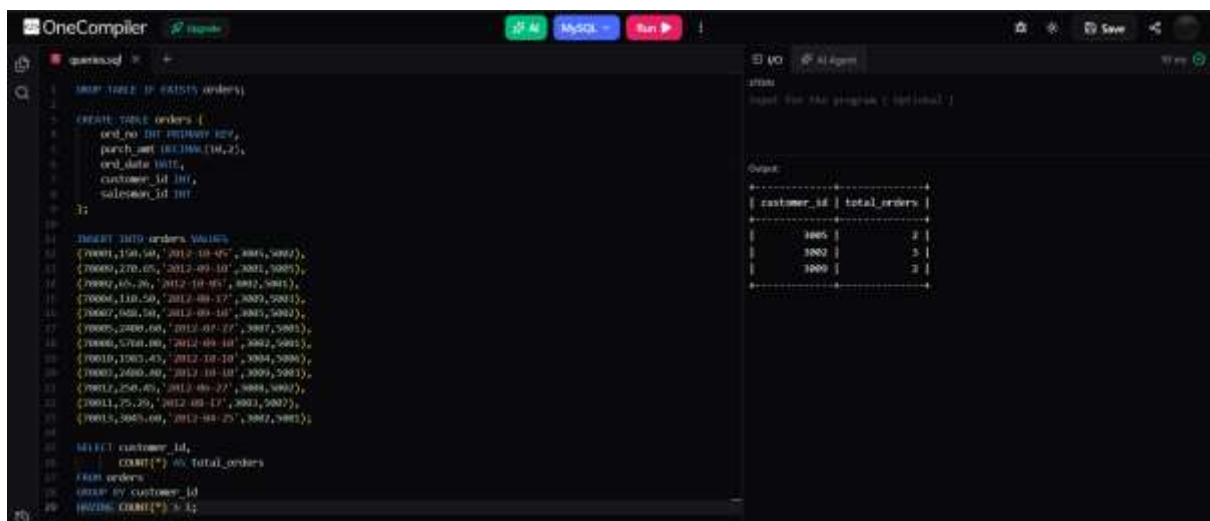
```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 ({70001,150.50,'2012-10-05',3005,5002}),
13 ({70005,278.65,'2012-09-18',3001,5005}),
14 ({70002,65.26,'2012-10-05',3002,5001}),
15 ({70004,110.50,'2012-08-17',3009,5003}),
16 ({70007,948.50,'2012-09-10',3005,5002}),
17 ({70005,2480.60,'2012-07-27',3007,5001}),
18 ({70008,5760.00,'2012-09-10',3002,5001}),
19 ({70010,1983.43,'2012-10-10',3004,5006}),
20 ({70003,2480.40,'2012-10-10',3009,5003}),
21 ({70012,250.45,'2012-06-27',3008,5002}),
22 ({70011,75.29,'2012-08-17',3003,5007}),
23 ({70013,3045.60,'2012-04-25',3002,5001});
24
25 SELECT customer_id,
26        SUM(purch_amt) AS total_sales
27 FROM orders
28 GROUP BY customer_id
29 HAVING SUM(purch_amt) BETWEEN 1000 AND 5000;
```

Output:

customer_id	total_sales
3005	1099.00
3009	2590.90
3007	2480.60
3004	1983.43

Q19 — COUNT + HAVING

COUNT() + HAVING — Find Customers with More Than One Order



```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 ({70001,150.50,'2012-10-05',3005,5002}),
13 ({70005,278.65,'2012-09-18',3001,5005}),
14 ({70002,65.26,'2012-10-05',3002,5001}),
15 ({70004,110.50,'2012-08-17',3009,5003}),
16 ({70007,948.50,'2012-09-10',3005,5002}),
17 ({70005,2480.60,'2012-07-27',3007,5001}),
18 ({70008,5760.00,'2012-09-10',3002,5001}),
19 ({70010,1983.43,'2012-10-10',3004,5006}),
20 ({70003,2480.40,'2012-10-10',3009,5003}),
21 ({70012,250.45,'2012-06-27',3008,5002}),
22 ({70011,75.29,'2012-08-17',3003,5007}),
23 ({70013,3045.60,'2012-04-25',3002,5001});
24
25 SELECT customer_id,
26        COUNT(*) AS total_orders
27 FROM orders
28 GROUP BY customer_id
29 HAVING COUNT(*) > 1;
```

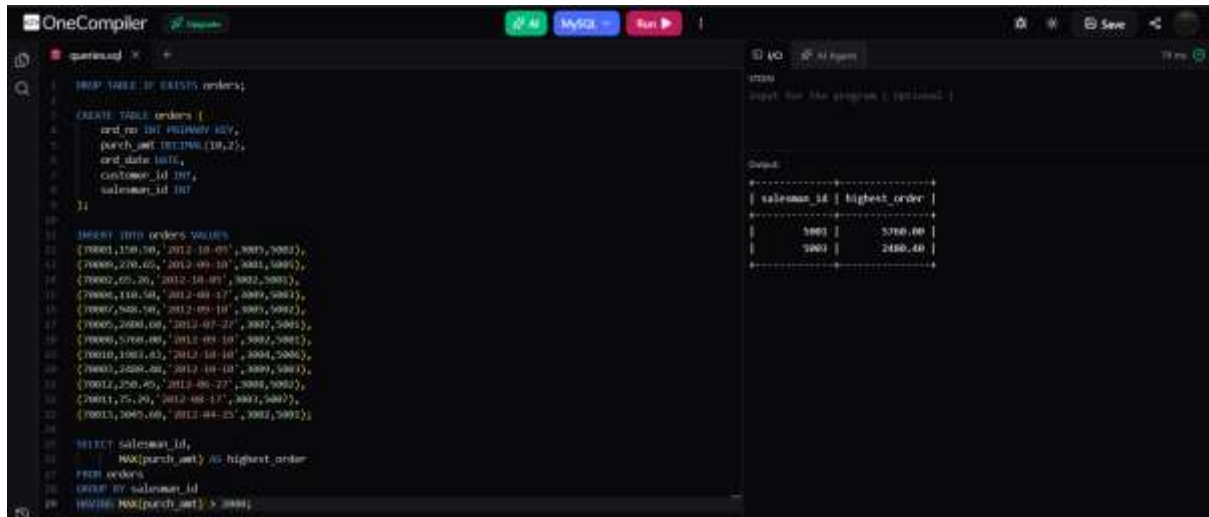
Output:

customer_id	total_orders
3005	2
3002	3
3009	2

Q20 — MAX + GROUP BY + HAVING

MAX() + GROUP BY + HAVING — Find Salesmen

Whose Highest Order Exceeds 2,000



The screenshot shows the OneCompiler MySQL IDE. The left pane contains a SQL script that creates an 'orders' table, inserts 13 rows of data, and then runs a query to find salesmen whose highest order exceeds 2,000. The right pane shows the output of the query as a table with two columns: 'salesman_id' and 'highest_order'.

```
1 DROP TABLE IF EXISTS orders;
2
3 CREATE TABLE orders (
4   ord_no INT PRIMARY KEY,
5   purch_amt DECIMAL(10,2),
6   ord_date DATE,
7   customer_id INT,
8   salesman_id INT
9 );
10
11 INSERT INTO orders VALUES
12 ({70001,1500.00,'2012-10-05',3005,3001}),
13 ({70009,2700.00,'2012-09-10',3001,3004}),
14 ({70002,650.00,'2012-10-05',3002,3003}),
15 ({70006,1100.00,'2012-08-17',3009,3002}),
16 ({70007,8450.00,'2012-09-10',3009,3002}),
17 ({70005,2400.00,'2012-07-27',3007,3002}),
18 ({70000,5700.00,'2012-09-10',3002,3002}),
19 ({70010,1900.00,'2012-10-10',3004,3006}),
20 ({70003,2200.00,'2012-10-10',3009,3003}),
21 ({70012,350.00,'2012-06-27',3008,3002}),
22 ({70011,75.00,'2012-06-17',3003,3002}),
23 ({70013,2000.00,'2012-04-25',3002,3002});
24
25 SELECT salesman_id,
26        MAX(purch_amt) AS highest_order
27 FROM orders
28 GROUP BY salesman_id
29 HAVING MAX(purch_amt) > 2000;
```

Output:

salesman_id	highest_order
3001	2700.00
3002	5700.00